

Rapport Projet OOP

TFT

Le projet est un **Auto-Battler** développé en C# s'exécutant dans la console. Il s'inspire directement du genre popularisé par *Teamfight Tactics* (TFT). Le principe repose sur le recrutement d'unités automatiques et la gestion stratégique de ressources. Contrairement à un jeu d'action, le joueur n'intervient pas directement pendant les combats ; son rôle est celui d'un tacticien qui prépare son armée avant chaque affrontement.

Règles principales

Le gameplay s'articule autour de deux phases distinctes :

1. **Phase de préparation : Gestion de l'économie** : Le joueur reçoit de l'or à chaque tour pour acheter des unités dans une boutique aléatoire.
 - **Achat/vente** : Le joueur peut acheter et vendre des champions dans une boutique
 - **Positionnement** : Les unités sont placées sur une grille de combat (représentée ici par des caractères textuelles).
 - **Synergies** : Le regroupement d'unités de la même classe ou origine octroie des bonus passifs à l'équipe.
 - **Évolution** : L'achat de plusieurs exemplaires d'une même unité permet de la faire monter en niveau (système de "stars"), augmentant ses statistiques.
2. **Phase de combat** :
 - Les unités s'affrontent de manière autonome selon un algorithme d'intelligence artificielle simple (ciblage de l'ennemi le plus proche, utilisation de capacités une fois la barre de mana remplie).
 - Le joueur perd des points de vie si son équipe est vaincue. Le dernier survivant remporte la partie.

Inspirations

L'inspiration majeure est **Teamfight Tactics (Riot Games)**, dont j'ai repris les piliers : l'aspect aléatoire de la boutique, la gestion du banc de touche et l'importance des synergies.

Structure et Anatomie d'un Champion dans TFT

Dans l'architecture de *Teamfight Tactics*, un champion est une entité complexe définie par quatre piliers principaux :

1. L'Identité et le Coût

Chaque champion possède un **nom** et appartient à une catégorie de **rareté** (souvent appelée "Tier", de 1 à 5). Cette rareté définit son coût en pièces d'or et sa probabilité d'apparition dans la boutique. Plus le coût est élevé, plus les statistiques de base et l'impact de sa capacité sont puissants.

2. Les Statistiques de Combat (Data)

Un champion est caractérisé par un set de données numériques qui évoluent selon son niveau d'étoiles (1, 2 ou 3 stars) :

- **Survie** : Points de vie (HP) et Armure / Résistance magique.
- **Offensive** : Dégâts d'attaque (AD), Vitesse d'attaque et Portée de tir (Range).
- **Mana** : Une jauge de départ et un seuil maximum. Le mana se régénère en infligeant ou en recevant des dégâts.

3. Les Traits (Synergies)

C'est l'aspect le plus stratégique. Chaque champion possède au moins deux traits (souvent une **Origine** et une **Classe**).

- *Exemple* : Un champion peut être "Cyborg" (Origine) et "Épéiste" (Classe).
- Ces traits ne sont actifs que si le joueur possède suffisamment de champions **différents** partageant le même trait sur le terrain.

4. La Capacité Spéciale (Spell)

Une fois sa barre de mana remplie, le champion déclenche une action unique. Cette capacité peut être :

- **Active** : Un sort de zone, un étourdissement (CC), ou un soin.
- **Passive** : Un bonus de statistiques permanent ou un effet déclenché toutes les trois attaques.

2. Simplifications

Le développement d'un *Auto-battler* complet est un défi technique majeur. Afin de garantir un produit fonctionnel et équilibré dans les délais impartis, plusieurs concessions ont été faites par rapport au titre original de Riot Games.

Gameplay et Système de combat

- **Combat au tour par tour :** Contrairement au temps réel simulé de TFT, les unités agissent ici de manière séquentielle. Cela permet une meilleure lisibilité dans un terminal et une gestion plus simple de la file d'actions en C#.
- **Adversaire unique :** Le système de lobby à 8 joueurs a été simplifié en un duel permanent contre une IA unique afin de se concentrer sur la logique de combat plutôt que sur le réseau ou la gestion complexe d'un tournoi.
- **Équilibrage par l'uniformisation :** Pour garantir l'équité sans une phase de test massive, toutes les unités utilisent le même système de calcul d'attaque basé sur leurs statistiques brutes. Cela évite l'émergence d'unités "brisées" (overpowered) dues à des mécaniques uniques non maîtrisées.

Économie et Évolution

- **Agnosticisme de la rareté :** Avec un pool réduit de **8 champions**, l'implémentation de niveaux de rareté (Tiers) aurait artificiellement bloqué la progression du joueur. Tous les champions ont donc la même probabilité d'apparition.
- **Système de Synergies ciblé :** J'ai choisi de n'implémenter que les **Origines** (au nombre de 4 : *Noble, Sauvage, etc.*). Les synergies de **Classes** (Guard, Assassin, Marksmann, Wizard) ont été écartées pour le moment afin de valider d'abord le fonctionnement algorithmique des bonus d'Origines.

Attributs et Statistiques des Champions

- **Élagage des caractéristiques :** Bien que les propriétés **Mana** et **Vitesse d'attaque** soient présentes dans le code (préparation pour le futur), elles ne sont pas exploitées dans cette version.
- **Simplification des calculs de dégâts :** La résistance magique a été fusionnée avec l'armure dans une statistique de **Défense** unique. De même, les capacités actives (Spells) ne sont pas déclenchées, le combat se concentrant sur les statistiques de base.

3. Étapes de conception

Le développement du projet a suivi une approche incrémentale, partant de la modélisation des données brutes vers la gestion complexe de la boucle de jeu.

Phase 1 : Modélisation et Héritage (Fondations)

La première étape a consisté à définir l'architecture des entités.

- **Hiérarchie des classes :** Mise en place de la classe mère `Champion` et déclinaison en classes filles par archétypes (`Guard`, `Marksmann`, `Wizard`, `Assassin`). Chaque classe a été instanciée avec deux champions ayant deux classes distinctes pour assurer un premier équilibre.
- **Système d'attributs :** Définition des origines (`Sauvage`, `Elfe`, `Noble`, `Ninja`) et des statistiques vitales. Initialement basées sur les données officielles de TFT, ces statistiques ont été ajustées pour s'adapter à l'environnement console.

Phase 2 : Gestion de l'espace et Robustesse

Une fois les entités créées, il a fallu définir leur environnement.

- **Structure de jeu :** Création de la map et du banc (zone de réserve). Cela incluait la logique de placement et de déplacement des unités.
- **Gestion des erreurs :** Implémentation de classes d'exceptions personnalisées (`GameRuleException`, `GameFoundException`) pour garantir la stabilité du programme et empêcher les actions invalides (ex: placer un champion sur une case occupée).

Phase 3 : Logique métier et Mécaniques avancées

C'est ici que le "moteur" du jeu a été développé.

- **Moteur Aléatoire :** Développement d'une classe générique utilisant des types génériques `<T>` pour gérer l'aléa (boutique, butin).
- **Système de Synergies :** Création du `GameManager` pour calculer dynamiquement les bonus d'origines présents sur la map et les appliquer aux statistiques des unités en temps réel.
- **Évolution des unités :** Implémentation du système de fusion (passage au niveau supérieur lors de l'obtention de trois unités identiques) et mise en place de la boutique.

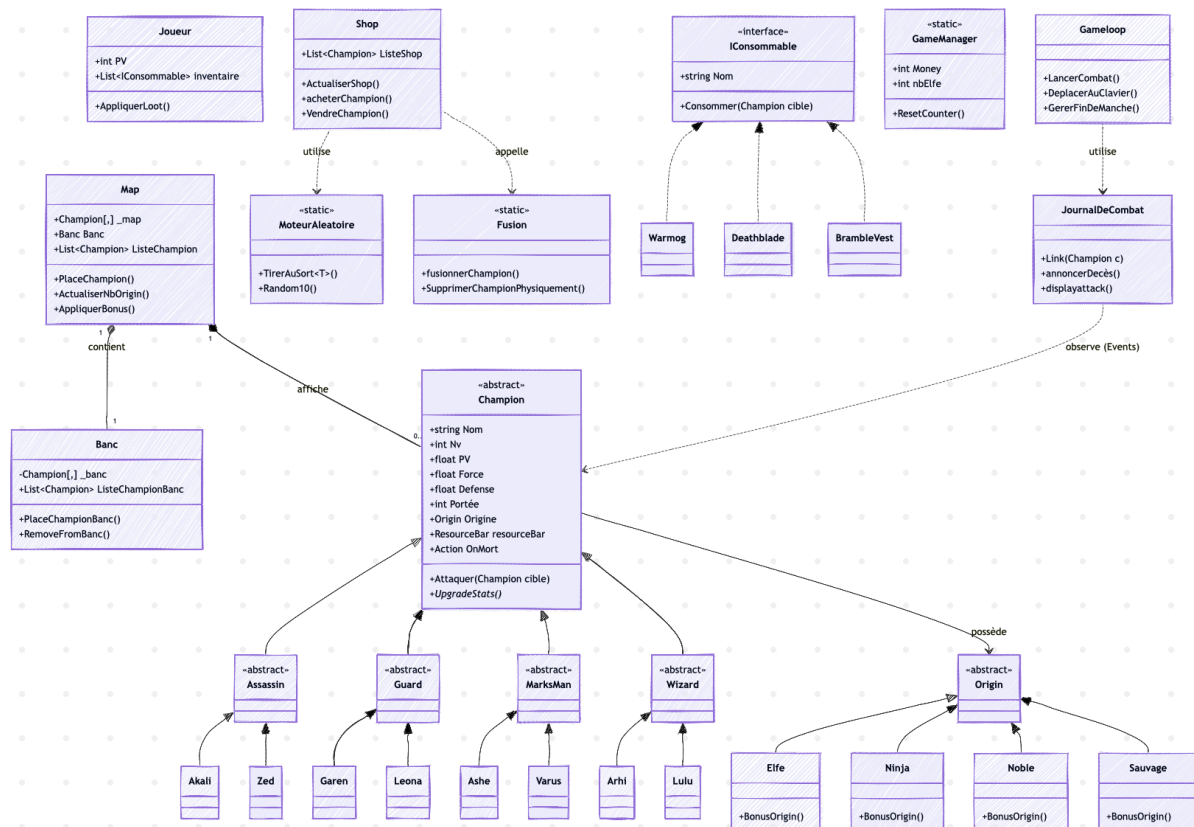
Phase 4 : Extensibilité et Finalisation

La dernière phase s'est concentrée sur l'expérience utilisateur et l'ouverture du code.

- **Découplage par les interfaces** : Utilisation d'interfaces pour le système d'objets équipables, permettant d'améliorer les statistiques de manière modulaire.
- **Gestion des Événements** : Mise en place d'un **Journal de Combat** utilisant les délégués C# (? . Invoke). Cela permet de séparer la logique de combat de l'affichage textuel, rendant le code plus propre et évolutif.
- **Boucle de jeu** : Assemblage final des phases de préparation et de combat dans une boucle principale (Game Loop) optimisée.

4. Architecture Logicielle

L'architecture de ce projet repose sur une approche orientée objet rigoureuse, visant à séparer la logique pure du jeu de l'affichage en console. Elle s'articule autour de quatre piliers majeurs.



A. La Hiérarchie des Champions (Héritage et Spécialisation)

Pour gérer la diversité des unités tout en évitant la duplication de code, j'ai implémenté une structure d'héritage à trois niveaux :

- **La Classe Abstraite `Champion`** : C'est le socle commun. Elle définit les attributs vitaux (HP, Force, Défense) et les comportements universels comme la recherche de cible (`TrouverCibleLaPlusProche`) et le système d'attaque.
- **Les Classes d'Archétypes (`Assassin`, `Guard`, `MarksMan`, `Wizard`)** : Ces classes intermédiaires permettent de spécialiser les statistiques de base. Par exemple, un `Guard` héritera nativement d'une défense plus élevée.
- **Les Classes d'Instances (`Akali`, `Garen`, etc.)** : Elles représentent les unités finales. Elles surchargent (`override`) la méthode `UpgradeStats()` pour définir précisément comment leurs statistiques évoluent lors d'une montée en niveau.

B. Gestion de l'Espace et Composition (`Map` & `Banc`)

Le moteur de jeu utilise le principe de **composition** pour structurer l'espace de jeu :

- **La `Map`** ne se contente pas d'être une grille ; elle possède une instance de la classe **`Banc`**. Cette structure permet de gérer les transferts d'unités (du banc vers le terrain) via des méthodes dédiées, garantissant que les listes de champions (`ListeChampion` et `ListeChampionBanc`) restent toujours synchronisées avec l'affichage visuel.
- **Le `GameManager`** fait office de "cerveau global" statique, stockant l'état de l'économie (`Money`) et le compteur de synergies sans avoir à passer ces données de classe en classe.

C. Découplage et Événements (`Le JournalDeCombat`)

L'un des points forts de l'architecture est l'utilisation des **délégués et événements C#** (`Action`).

- Plutôt que d'écrire des `Console.WriteLine` directement dans la logique d'attaque ou de mort des champions, ces derniers "notifient" le système via des événements (`OnMort`, `OnAttaquer`).
- La classe **`JournalDeCombat`** s'abonne à ces événements via la méthode `Link()`. Cela permet de séparer totalement la mécanique de jeu de la présentation textuelle : on pourrait changer tout l'affichage du jeu sans jamais modifier une seule ligne de la classe `Champion`.

D. Algorithmique et Généricité

Le projet utilise des outils avancés pour assurer sa flexibilité :

- **`MoteurAleatoire`** : Utilise la **généricité** (`<T>`) pour tirer au sort n'importe quel type d'objet (`Champion` ou `Item`). J'y ai intégré un système de **clonage**

(`MemberwiseClone`) pour s'assurer que lorsqu'un champion est acheté dans le shop, il s'agisse d'une nouvelle instance indépendante.

- **Fusion** : Cette classe statique gère la montée en puissance de manière récursive. Lorsqu'un champion est acheté, le système scanne l'inventaire total et, s'il trouve trois doublons, déclenche la fusion. Le processus est capable de s'appeler lui-même pour vérifier si une fusion de niveau 2 peut déclencher une fusion de niveau 3.

E. Extensibilité via les Interfaces

Le système de butin (Loot) repose sur l'interface **IConsommable**. Chaque objet (`Warmog`, `Deathblade`) implémente sa propre méthode `Consommer()`. Cela permet au joueur de posséder un inventaire d'objets aux effets radicalement différents sans que la classe `Joueur` n'ait besoin de connaître le détail technique de chaque item.

6. Travail futur et pistes d'amélioration

Bien que le projet soit fonctionnel et réponde aux exigences du cahier des charges, l'architecture a été pensée pour permettre des évolutions majeures. Voici les axes de développement envisagés pour enrichir l'expérience de jeu.

A. Évolutions des mécaniques de Gameplay

- **Activation du système de Mana et Sorts** : Les fondations techniques (classe `ResourceBar` et propriétés dans `Champion`) sont déjà présentes. Le travail futur consisterait à implémenter la méthode `LancerSort()` pour chaque unité. Cela ajouterait une dimension tactique cruciale, où le placement permettrait de charger son mana plus ou moins vite.
- **Implémentation des Synergies de Classes** : Actuellement, seules les **Origines** octroient des bonus. L'ajout des synergies pour les **Classes** (`Guard`, `Assassin`, etc.) permettrait une double profondeur stratégique, forçant le joueur à faire des choix d'équilibre plus complexes.
- **Système d'objets permanents** : Passer des objets consommables à un système d'inventaire d'équipement. Les objets pourraient être portés par les champions, combinés pour créer des artefacts plus puissants, et récupérés lors de la vente d'une unité.

B. Intelligence Artificielle et Équilibrage

- **IA de positionnement** : Pour l'instant, l'adversaire est généré de façon aléatoire. Une amélioration majeure serait de coder une IA capable d'analyser le placement du joueur pour positionner ses "tanks" face aux "carries" adverses, simulant ainsi un véritable défi tactique. Il faudrait aussi représenter l'ennemi par une vraie classe mais par souci de temps je n'ai pas pu le faire .
- **Algorithme d'équilibrage dynamique** : Utiliser les données des combats pour ajuster automatiquement les statistiques des champions les moins utilisés, garantissant qu'aucune "meta" ne domine le jeu.

C. Améliorations Techniques et Visuelles

- **Transition vers une Interface Graphique (GUI)** : Le moteur est actuellement en mode texte. Grâce au découplage réalisé avec le `JournalDeCombat` et les événements, il serait possible de porter le projet sur un moteur comme **Unity** ou **Raylib** sans modifier la logique métier. On remplacerait simplement les symboles ASCII par des sprites et des animations.
- **Implémentation de tous les évènements** : Actuellement, il reste encore beaucoup de `console.writeline`, pour pouvoir intégrer correctement le jeu dans unity il faudrait créer plus d'évènements .
- **Mode Multijoueur "Ghost Board"** : Implémenter un système où le joueur affronte les compositions enregistrées d'autres joueurs, créant une impression de compétition asynchrone fidèle au genre Auto-battler.